Internal Task Portal · Part 2 · design only, not implemented

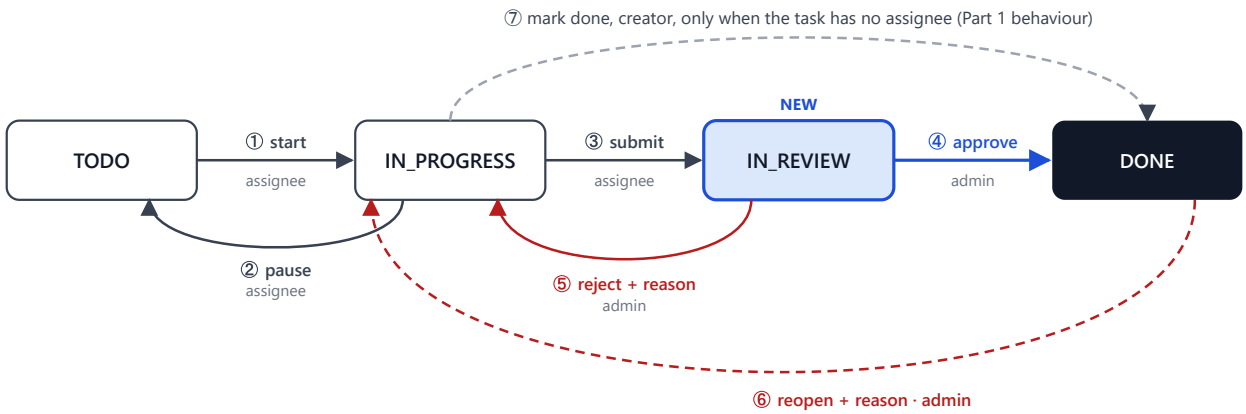
Task Portal Part 2: process flow and sprint plan

> **Goal:** users can filter tasks by status and assignee and open a task details page. The assignee uploads evidence that the task is complete, and an admin approves it. Part 1 keeps working as before.

1 Assumptions

- A task has **one assignee**, who must be an existing user. The creator or an admin assigns or reassigns it.
- A task is visible to its **creator, its assignee and admins**. Anyone else gets 404, as in Part 1.
- **Admin** means `User.role = ADMIN`. The column already exists from Part 1.
- **Evidence:** PDF, PNG, JPG or DOCX, up to 10 MB, stored privately and downloaded through short-lived links.
- An **unassigned task** behaves as in Part 1: its creator can mark it done without review.

2 Task statuses



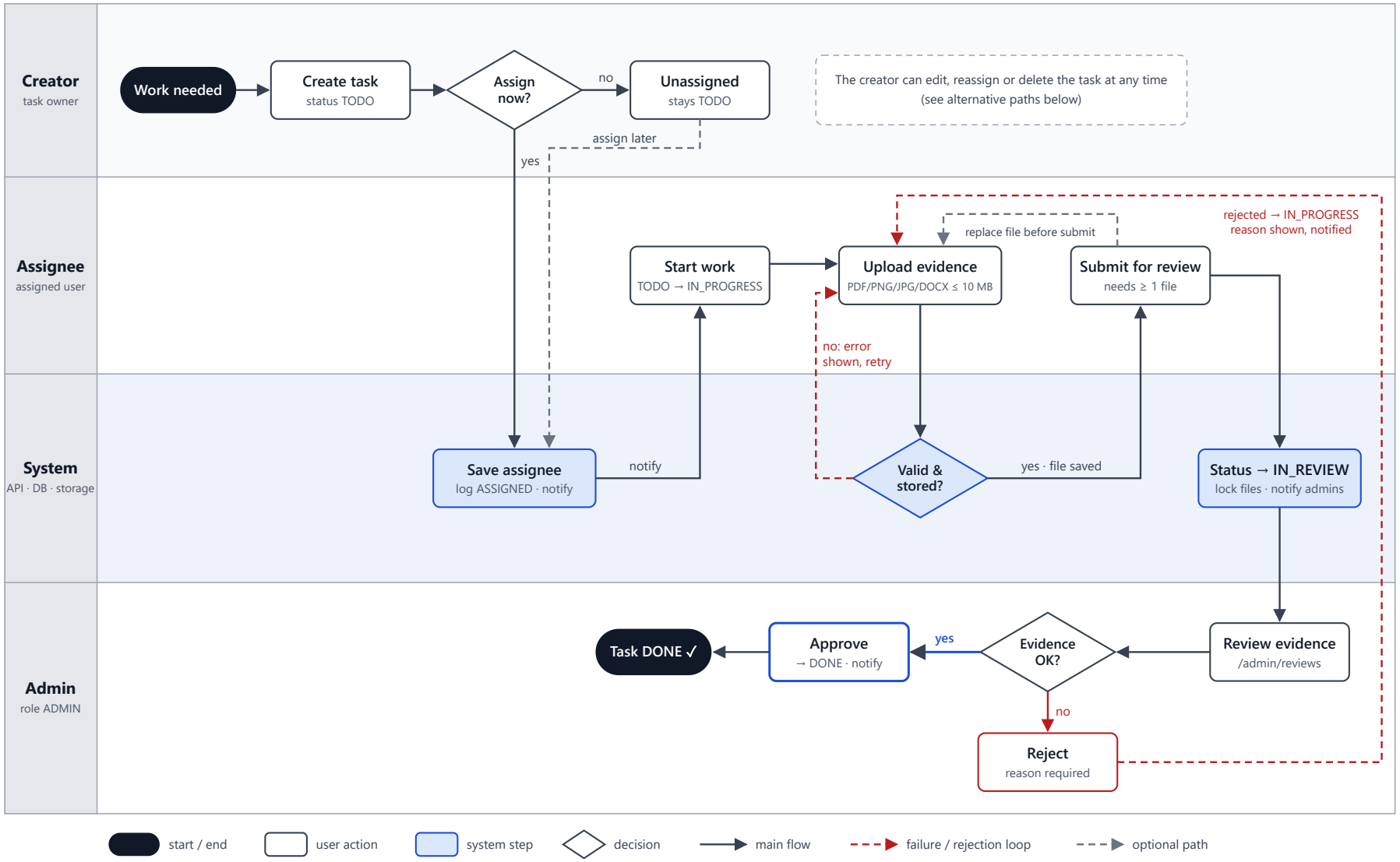
3 Who does what

Step	Who
Create, edit, delete	Creator (admins can also delete)
Assign / reassign	Creator or admin
Start work, upload evidence, submit	Assignee
Approve / reject / reopen	Admin (not on a task assigned to them)

4 Key technical changes

- New status `IN_REVIEW`. The server enforces every status change.
- New tables: `Evidence` (file details) and `TaskEvent` (history: who did what, and rejection reasons).
- One **access-rules module** replaces Part 1's "own tasks only" checks.
- **Sessions that can be revoked**, so a role change takes effect straight away.

5 Process flow map: from creation to completion



Upload fails (wrong type, too big, network error): an error is shown, nothing is saved, and the user retries.

Assignee deleted: the task becomes unassigned and the creator reassigns it.

Replace a file before submitting: the new file replaces the old one, and the old one stays in the history.

Admin rejects: the task goes back to IN_PROGRESS with the reason shown, and the loop repeats until approved.

Reassigned mid-work: the new assignee continues, and the old one loses access. Not allowed while in review.

No access: 404, the same as a missing task. Allowed to see but not to act: 403.

6 Sprint plan

Team (manpower): 2 full-stack developers for 4 weeks. **Dev A** covers backend, data and security; **Dev B** covers frontend and pages. **QA** writes the test plan in sprint 1 and runs end-to-end tests in sprint 2; **design** spends 2–3 days in week 1 on the details, upload and review screens.

Story points: Fibonacci scale, 1 point ≈ half a dev-day. Capacity is about 16 points per developer per sprint, leaving room for code review and meetings. Sprint 2 is planned lighter (22 points) to leave a buffer for bug fixes and stakeholder testing. **Blue** = critical path.

#	Ticket	Done when	Pts	Owner
Sprint 1: foundations, assignment, filters, details page — 31 pts · Dev A 16 · Dev B 15				
1	DB changes: `IN_REVIEW` status, Evidence and TaskEvent tables, Session table, indexes	Migration runs on a copy of production with no data change	3	Dev A
2	Access-rules module (creator / assignee / admin) + full tests	Another user always gets 404; all Part 1 tests pass	8	Dev A
3	Revocable sessions (session id in the token, checked against the DB)	Logout and role changes take effect on the next request	5	Dev A
4	Assign / reassign: API + user picker in the task dialog	Assignee sees the task in their list	5	Dev B
5	Filters by status and assignee (in the URL, applied on the server)	Filtered link can be shared; never shows hidden tasks	3	Dev B
6	Task details page `/tasks/[id]`	Opens from the list; no access → 404	5	Dev B
7	Storage choice: Vercel Blob vs S3 (1-day check)	Private 10 MB upload works in a prototype	2	Dev B
Sprint 2: evidence, review, release — 22 pts + 5 stretch · Dev A 10 (+5) · Dev B 7 · QA + Dev B 5				
8	Evidence upload: straight to storage, type and size checked, replace before submit	Only the assignee can upload; files are never public	8	Dev A
9	Submit for review (IN_PROGRESS → IN_REVIEW)	Blocked without a file; files locked after submitting	2	Dev A
10	Admin review queue `/admin/reviews` + approve / reject with reason	Approve → DONE, reject → IN_PROGRESS; non-admins get 404	5	Dev B
11	Task history on the details page	Shows who did what, and rejection reasons	2	Dev B
12	End-to-end tests for the full flow + release behind a feature flag	Full flow passes in CI; flag can be turned off safely	5	QA + Dev B
13	*Stretch:* email notifications (assigned, submitted, approved, rejected)	Never blocks the user action; one email per event	5	Dev A

Critical path: 1 → 2 → 8 → 9 → 10 → 12. Key dependencies: 2 needs 1; tickets 4–6 need 2; 8 needs 6 and 7; 10 needs 3 and 9. Ticket 7 must finish in week 1 so that 8 can start on time. Total: 53 points, plus 5 stretch.

7 Top risks

- **Access rules leak other users' tasks.** Mitigation: one module, fully tested, built first.
- **Large uploads hit serverless limits.** Mitigation: upload straight to storage; the server only saves file details.
- **Two admins review at once.** Mitigation: the first decision wins, and the second admin gets a clear error.

8 Key trade-offs

- **One status field** with `IN_REVIEW`, not a separate review state: simpler filters and UI.
- **Rejection returns the task to IN_PROGRESS** with a reason, instead of a new status.
- **Filters live in the URL and run on the server:** shareable, and they scale as lists grow.
- **API routes, not Server Actions,** to stay consistent with Part 1.